Семинар 2. Часы и время в распределенных системах

Распределенные вычисления

Георгий Семенов

Институт прикладных компьютерных наук
Университет ИТМО

весна 2026

Организационные моменты

- Материалы семинаров и домашек появились на GitHub
- Есть вероятность, что первая домашка будет посвящена часам! (о них – сегодня)

Содержание

1 (!) Ликбез навстречу Docker и Linux

2 Линейные часы и время в распределенных системах

3 Частично упорядоченное время и логические часы

Как исполняется программа в ОС?

- Операционная система – управляет ресурсами вычислительного узла и предоставляет API *системных вызовов* для программ
  - Поток – единица диспетчеризации для разделения CPU (int main())
  - Виртуальная память – пространство динамически выделяемой памяти RAM (+swap) для программы (malloc/free)
  - Сокет (ip + порт) – канал для обмена данными в рамках I/O

Как исполняется программа в ОС?

- Операционная система – управляет ресурсами вычислительного узла и предоставляет API *системных вызовов* для программ
  - Поток – единица диспетчеризации для разделения CPU (int main())
  - Виртуальная память – пространство динамически выделяемой памяти RAM (+swap) для программы (malloc/free)
  - Сокет (ip + порт) – канал для обмена данными в рамках I/O
- Что происходит, когда мы запускаем программу в Linux?
  - Создание подпроцесса (fork) – рождение нового PID как копии родительского процесса
  - Загрузка исполняемого файла в RAM для нового процесса (exec, секции ELF)
  - При этом был резервирована память под stack, а динамическая память (heap) будет выделяться по мере необходимости с помощью системных вызовов
  - Работа с памятью файлами, динамическими библиотеками, сокетами и т.п. – все через системные вызовы ОС (calling conventions)

Какие неявные зависимости есть у программы?

- Порядок сборки – библиотеки, пакеты, интерфейсы и API ОС
- Исполняющая среда – непосредственно ОС, виртуальная машина, интерпретатор и т.п.
- Файлы, появляющиеся в процессе работы программы
  - Stateful файлы – конфигурация, данные и т.п.
  - Временные и мусорные файлы – AppData, /tmp, /.cache и т.п.
- Сетевое окружение
  - Открытые порты (сокеты) и их forwarding наружу
  - Доступные IP-адреса сторонних сервисов и способ их адресации с помощью DNS

Что такое Docker и зачем он нужен?

Docker – платформа для исполнения приложений в изолированных *контейнерах*.

- Файл Dockerfile – декларативное описание процесса сборки образа (image) для контейнера
- В результате сборки образа по Dockerfile мы получаем *образ* (image), или несколько образов в случае multi-stage
- Мы можем запустить контейнер на основе образа, указывая дополнительные параметры:
  - Отображение портов внутри контейнера на порты хоста (forwarding)
  - Режимы сети (host, bridge, none) для обеспечения сетевой связности между контейнерам, хостом и внешним миром
  - Volumes – можно примонтировать директории на хосте внутрь контейнера для обмена данными между ними (данные не удалятся, когда контейнер будет удален)
  - Переменные окружения (environment) для передачи конфигурации внутрь контейнера

Пример Dockerfile

```
# ARG - Use build-time variables.
ARG RELEASE
# LABEL - Add metadata to an image.
LABEL org.opencontainers.image.ref.name=ubuntu
# ADD - Add local or remote files and directories.
ADD file ... in /
# CMD - Specify default commands.
CMD ["/bin/bash"] # Specify default commands.
# ENV - Set environment variables.
ENV JAVA_HOME=/opt/java/openjdk
# RUN - Execute build commands.
RUN /bin/sh -c set -eux;
# COPY - Copy files and directories.
COPY --chmod=755 entrypoint.sh /__cacert_entrypoint.sh
# ENTRYPOINT - Specify default executable.
ENTRYPOINT ["/__cacert_entrypoint.sh"]
```

Пример Dockerfile (продолжение)

```
# CMD - Specify default commands.
CMD ["jshell"]
# EXPOSE - Describe which ports your application is listening o
EXPOSE [25565/tcp]
# VOLUME - Create volume mounts.
VOLUME [/data]
# WORKDIR - Change working directory,
WORKDIR /data
# STOPSIGNAL - Specify the system call signal for exiting a con
STOPSIGNAL SIGTERM
# HEALTHCHECK - Check a container's health on startup.
HEALTHCHECK &{["CMD-SHELL" "mc-health"] "30s" "0s"
```

Как собрать образ и запустить контейнер?

```
# . - директория, где лежит Dockerfile, т.е. текущая
docker build -t minecraft-server .

# посмотреть локальные образы
docker image ls

# -d - запуск в фоновом режиме (detached)
# -p - port forwarding: отображаем порт 25565 на хост
docker run -d -p 127.0.0.1:25565:25565 minecraft-server

# посмотреть контейнеры
docker ps
```

Пример docker-compose.yaml

```yaml
services:
  minecraft: # название контейнера (сервиса)
    image: itzg/minecraft-server:latest # имя образа и тэг
    pull_policy: daily # как часто обновлять образ
    tty: true
    stdin_open: true
    ports: # отображаем внутр. порт 25565 на порт хоста 25565
      - "25565:25565"
    environment: # конфигурируем через переменные окружения
      EULA: "TRUE"
      VERSION: "1.21.11"
      SERVER_NAME: "Уютные зимние приключения"
      DIFFICULTY: normal
    volumes: # монтируем директорию на хосте внутрь
             # контейнера для сохранения данных
      - ./data:/data
```

Как собрать образ и запустить контейнер?

```
# в директории, где лежит docker-compose.yaml
docker compose up -d

docker compose stop # (1) просто остановим все
docker compose down # ИЛИ (2): еще и грохнем контейнеры и т.п.
```

Discussion: что для нас Docker?

- Docker - среда виртуализации для исполнения программ в изолированных контейнерах
- Внутри каждого контейнера запущена своя ОС (обычно Linux)
  - Ядро ОС резидентно, т.е. должно лежать в оперативной памяти хоста, поэтому важен размер
  - Alpine Linux – 5 MB по образу
- Docker свойственно захватывать ресурсы хоста
  - RAM - эмулирует виртуальную память для контейнера, выделяя её из общей памяти хоста
  - Firewall - Docker управляет iptables в Linux (использовать совместно с ufw - не стоит)
- Какие плюсы и минусы Docker?
  - (+) Декларативная абстракция для описания сред для сборки и запуска программ
  - (-) Ненулевые издержки на RAM

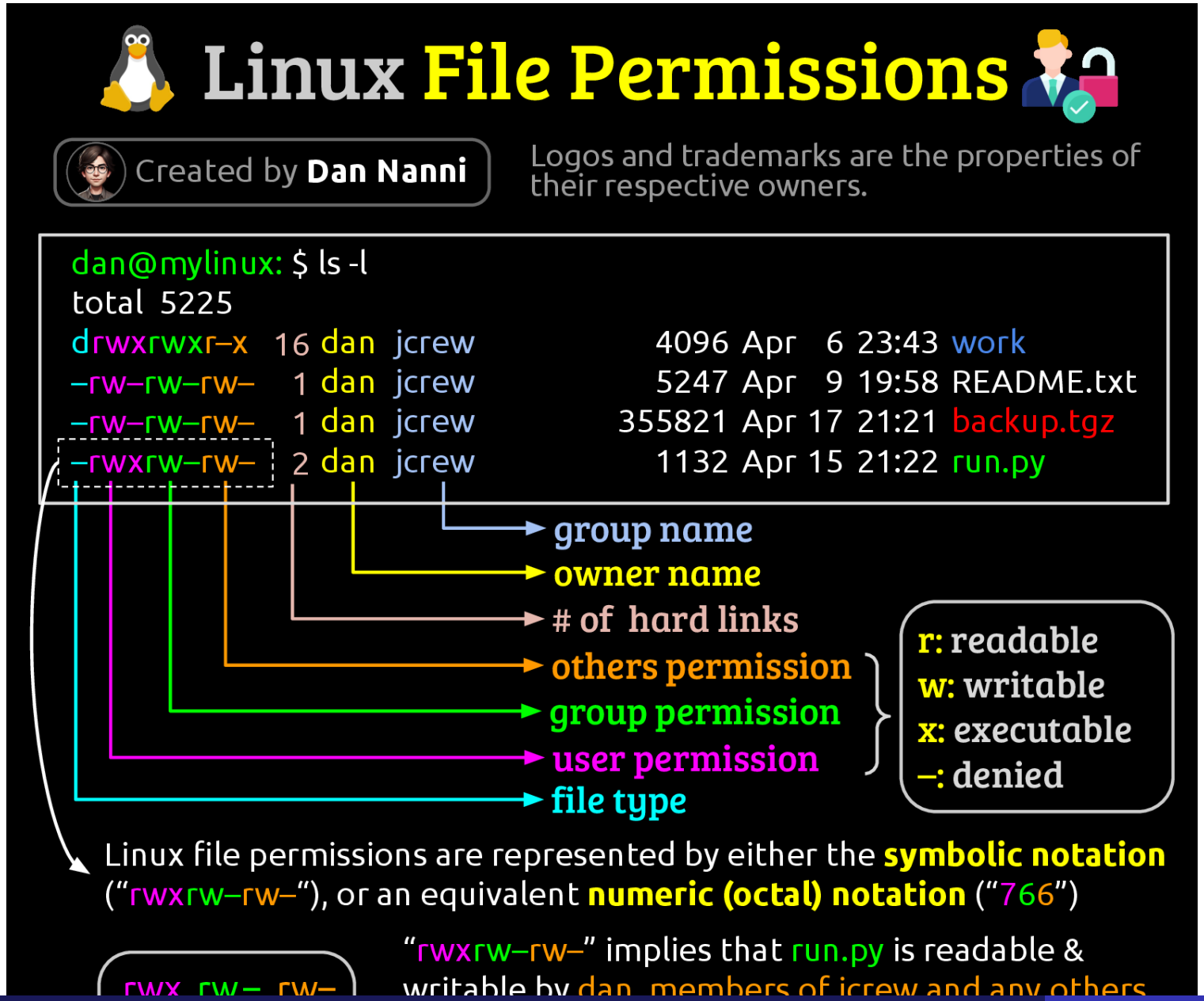
Какие свойства мы имеем в виду, говоря о Linux?

- Свободный, модульный и открытый код ОС и программ (copy-left лицензия GPL)
- Де факто промышленный стандарт: POSIX-совместимость, стандартные C-headers, ELF, .so и т.п.
- Декларативные способы решения задач по системному администрированию (в т.ч. DevOps)

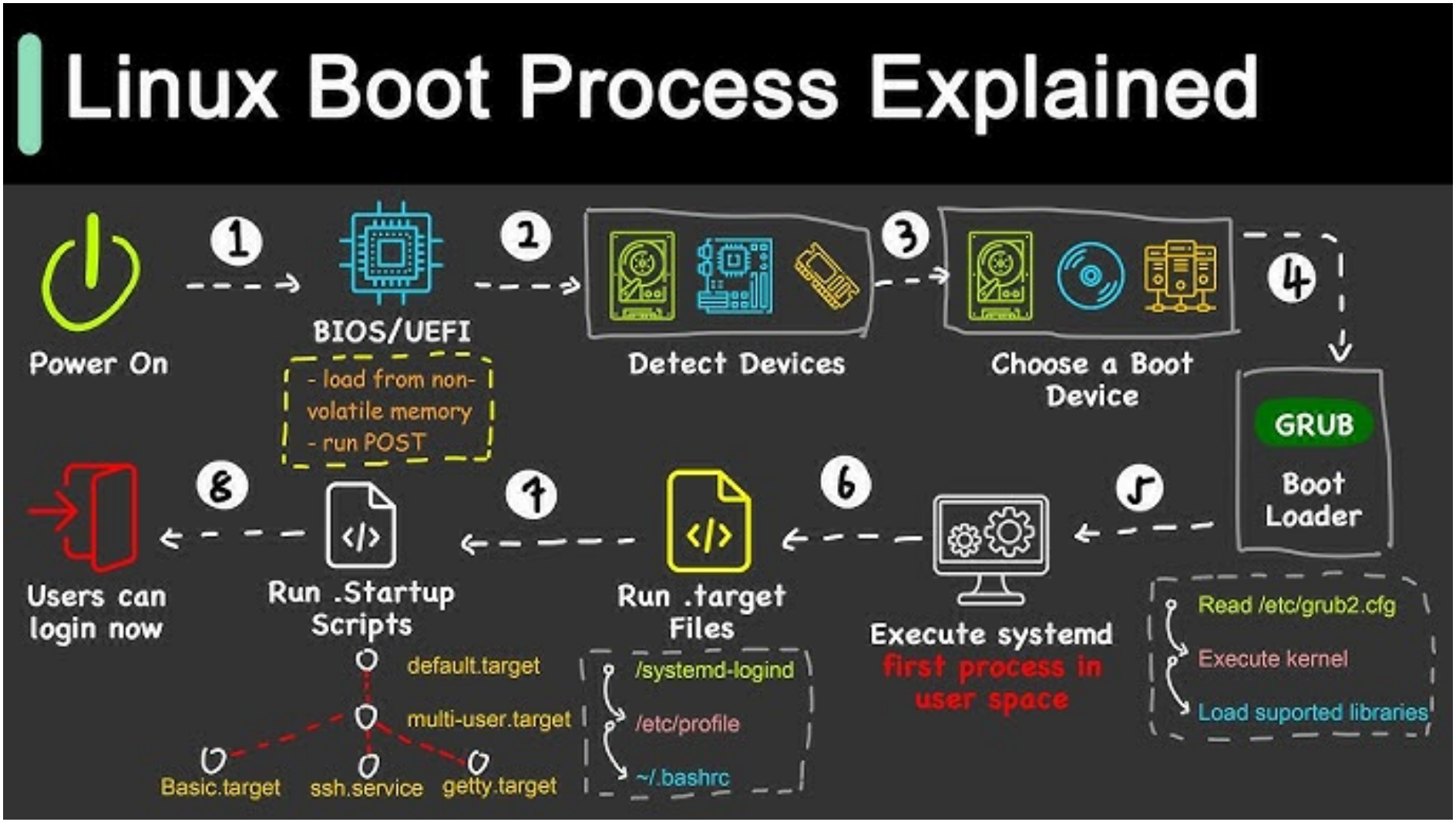
Что нужно обязательно представлять о Linux?

- Файловая система в Linux-системах
  - Все – это файл (включая устройства, сокеты и т.п.)
  - Иерархическая структура файловой системы (root, /home, /dev, /proc и т.п.)
  - Узлы иерархической структуры - inodes, ссылаются на блоки данных
  - Права доступа к файлам: владелец (u), группа (g), остальные (o); чтение (r), запись (w), выполнение (x)
- Порядок инициализации и управления ОС
  - Базово о физическом запуске - BIOS/UEFI, загрузчик (GRUB), ядро ОС, init-система (systemd)
  - Service в Linux: юнит, который может быть запущен, остановлен, перезапущен и т.п. с помощью systemctl
  - Процессы и демоны – top, htop, ps, systemctl status и т.п.
  - Пользователи и группы - useradd, groupadd, passwd и т.п.

Управление доступом к файлам



Загрузка сервисов



Упражнения (опциональные)

1. Создать папку (mkdir, -p) с файлами (touch, nano) и гранулярно настроить права доступа к ним (chmod, chown) под произвольную бизнес-задачу (например, для файлов веб-сервера - менять можем мы, но не он)
2. Создать service (service-файл) в Linux, который будет запускаться в качестве демона проверки состояния системы (скрипт на Bash) и логировать раз в какое-то время результаты в файл (systemctl list-dependencies, status, logs и т.п.)
3. Установить Docker, запустить произвольный сервис с помощью docker-compose:
   - nginx-сервер (проверить можно, открыв в браузере http://localhost:80)
   - Postgres (проверить, подключившись через psql, DataGrip и т.п.)
   - Minecraft-сервер (проверить можно, если есть игра)

Содержание

1. (!) Ликбез навстречу Docker и Linux
2. Линейные часы и время в распределенных системах
3. Частично упорядоченное время и логические часы

Время как проблема

- В распределённой системе нет глобальных часов: разные узлы, разные ядра процессора, разный «пульс времени»
- Время передачи сообщений в общем случае неизвестно и может быть большим (пример с ping)
- Еще хуже – физические часы дрейфуют ($\pm\epsilon$)

Вопрос: как определить, какое событие произошло раньше? В идеальном случае хочется:

- Монотонность – «время течет всегда вперед»
- Согласованность – если a произошло до b, то $t(a) < t(b)$
- Уникальность – разные события имеют различимые метки времени

Два типа времени

Тотально упорядоченно (total order, linear, wall-clock, linerializble)

Любые два события сравнимы $<$: до или после (иррефлексивное, транзитивное, антисимметричное).

Частично упорядоченное (partial order, causal, logical, concurrent)

Некоторые события несравнимы $<$: до, после или конкурентно (иррефлексивное, транзитивное, антисимметричное).

Физическое время $\Rightarrow$ тотальный порядок
Причинность $\Rightarrow$ частичный порядок

Real-Time Clock

Модель реального времени (физические часы) – это функция $C(t)$, которая отображает реальное время t в метку времени C.

$$C(t) = t + \epsilon(t)$$

где $\epsilon(t)$ — ошибка.
Два параметра:

- **Drift** — отклонение скорости (разность тактовой частоты от эталона)
- **Skew** — разница между часами (относительный сдвиг)

Синхронизация часов

Cristian's algorithm (1989)

- Осуществляем Round-Trip с запросом времени
- Вычисляем сдвиг как временной сдвиг между клиентом и сервером: $RTT/2$
- В чем недостаток алгоритма?

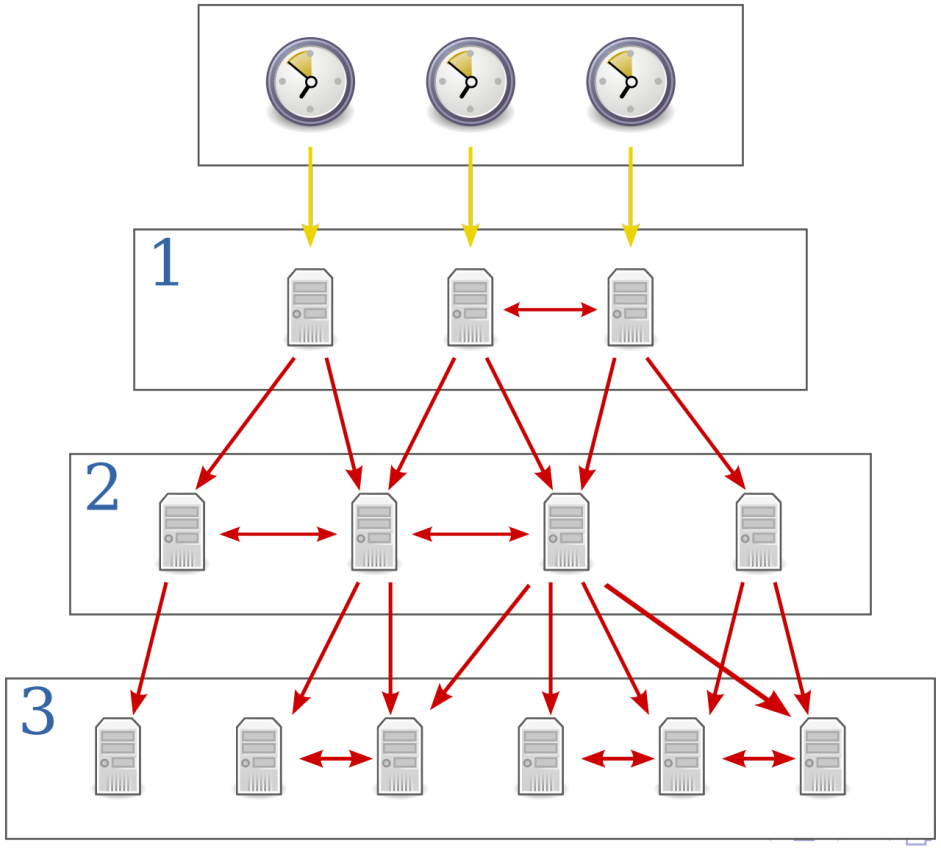
Berkeley algorithm (1989)

- Делегируем быть «часовщиком» одному узлу системы (через leader election)
- Как в прошлом алгоритме, собираем и оцениваем время на узлах (включая себя) через Round-Trip
- Усредняем и сообщаем всем узлам, какой offset им надо к себе прибавить

NTP

Network Time Protocol – семейство алгоритмов с поддержкой масштабируемости:

- Round-Trip с оценкой корректировки (алгоритм Marzullo)
- Иерархия страт (stratum 0 – эталонные часы)
- Точность – миллисекунды (Интернет), микросекунды (LAN)



Monotonic Clock

Требование:

$$t_2 > t_1 \Rightarrow C(t_2) \geq C(t_1)$$

Используется для:

- измерения интервалов
- таймаутов
- backoff

Модель времени в C++

`std::chrono::system_clock`

- отражает wall-clock
- может изменяться назад

`std::chrono::steady_clock`

- монотонный
- не связан с календарным временем
- не гарантирована синхронизация между машинами

TrueTime (Google Spanner)

Идея:

$$TT.now() = [earliest, latest]$$

Время задаётся интервалом неопределённости.
Позволяет:

- обеспечить внешнюю консистентность
- реализовать commit-wait

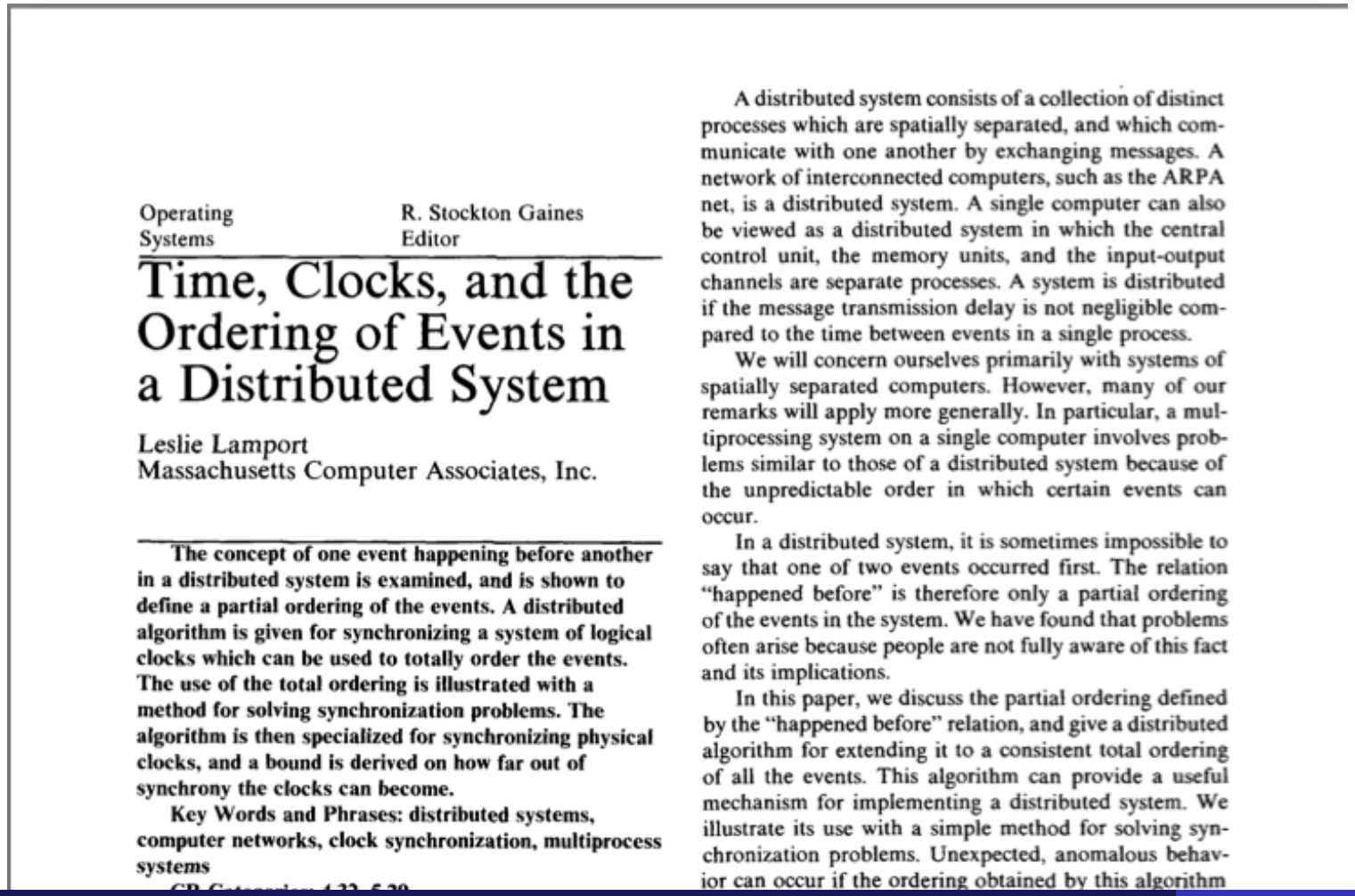
Содержание

1. (!) Ликбез навстречу Docker и Linux
2. Линейные часы и время в распределенных системах
3. Частично упорядоченное время и логические часы

Lamport (1978)

Статья: *Time, Clocks, and the Ordering of Events in a Distributed System*
Главная идея:

Время = причинность



Operating Systems — R. Stockton Gaines, Editor

Time, Clocks, and the Ordering of Events in a Distributed System

Leslie Lamport
Massachusetts Computer Associates, Inc.

The concept of one event happening before another in a distributed system is examined, and is shown to define a partial ordering of the events. A distributed algorithm is given for synchronizing a system of logical clocks which can be used to totally order the events. The use of the total ordering is illustrated with a method for solving synchronization problems. The algorithm is then specialized for synchronizing physical clocks, and a bound is derived on how far out of synchrony the clocks can become.

Key Words and Phrases: distributed systems, computer networks, clock synchronization, multiprocess systems

A distributed system consists of a collection of distinct processes which are spatially separated, and which communicate with one another by exchanging messages. A network of interconnected computers, such as the ARPA net, is a distributed system. A single computer can also be viewed as a distributed system in which the central control unit, the memory units, and the input-output channels are separate processes. A system is distributed if the message transmission delay is not negligible compared to the time between events in a single process.

We will concern ourselves primarily with systems of spatially separated computers. However, many of our remarks will apply more generally. In particular, a multiprocessing system on a single computer involves problems similar to those of a distributed system because of the unpredictable order in which certain events can occur.

In a distributed system, it is sometimes impossible to say that one of two events occurred first. The relation "happened before" is therefore only a partial ordering of the events in the system. We have found that problems often arise because people are not fully aware of this fact and its implications.

In this paper, we discuss the partial ordering defined by the "happened before" relation, and give a distributed algorithm for extending it to a consistent total ordering of all the events. This algorithm can provide a useful mechanism for implementing a distributed system. We illustrate its use with a simple method for solving synchronization problems. Unexpected, anomalous behavior can occur if the ordering obtained by this algorithm

Отношение happens-before

Определение:
$a \to b$, если:

1. События в одном процессе и a раньше b
2. a — отправка, b — получение сообщения
3. Транзитивность

Тотально упорядоченное время

$$t(a) < t(b) \Leftrightarrow a \to b$$

Частично упорядоченное время

$$t(a) < t(b) \Rightarrow a \to b$$

Часы Лампорта

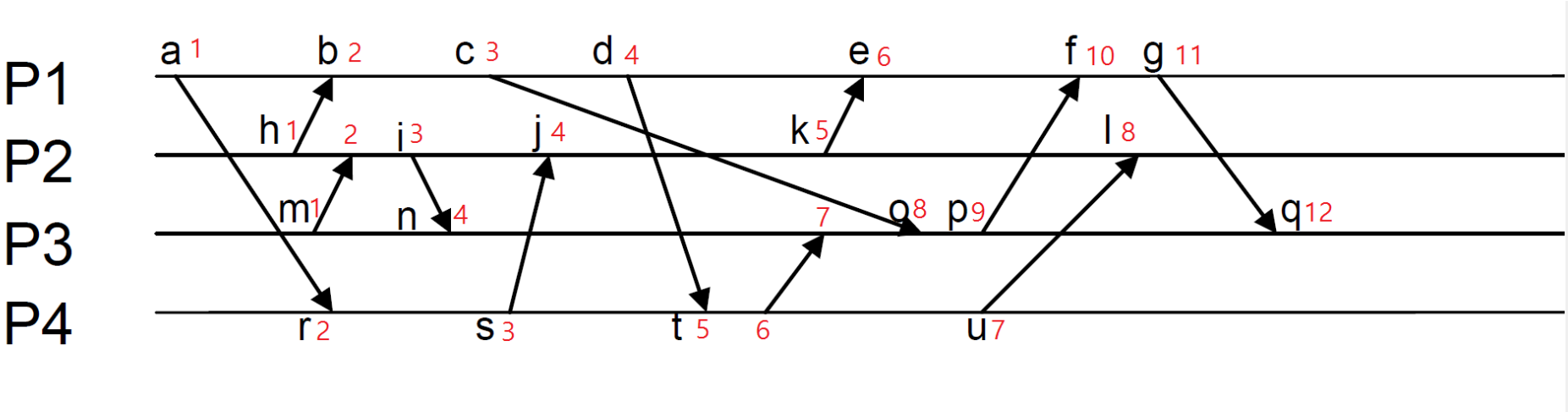
Интуиция:

- "ничто на Земле не проходит бесследно"
- если произошло B, и A happens-before B, то это должно привести к «утечке» времени, т.е. $L(A) < L(B)$

Алгоритм для каждого процесса P с локальной переменной C:

1. Когда хочу создать событие:
   - Добавлю в событие метку, соответствующую моим текущим часам: $L := C$
   - Атомарно инкрементирую свои часы на единицу: $C := C + 1$
2. Когда получаю чужое событие:
   - Обновляю свои часы: $C := \max(C, C_m) + 1$

Иллюстрация часов Лампорта

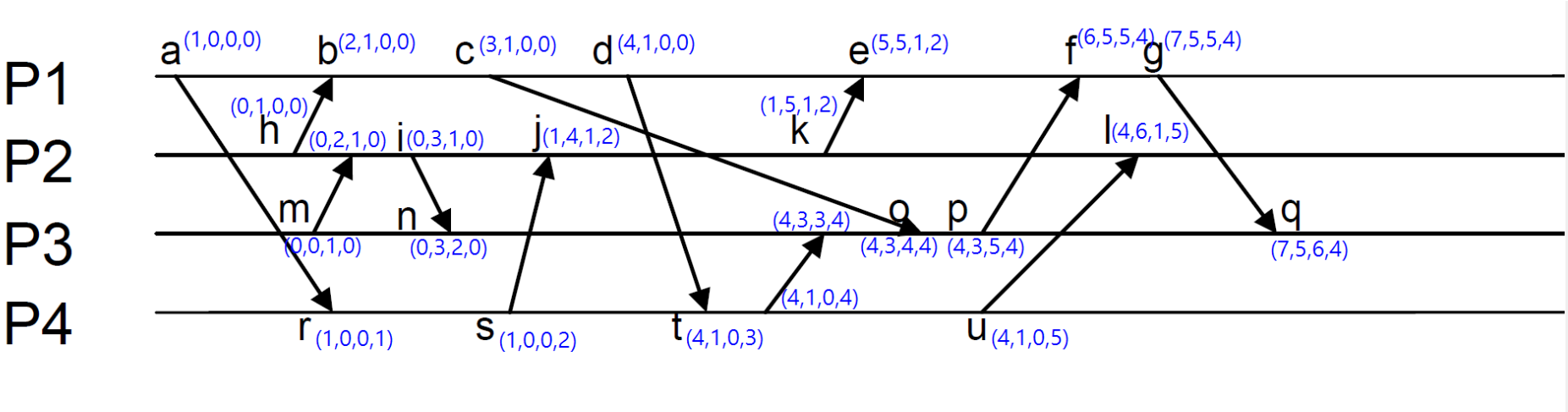


Векторные часы

Каждый процесс хранит вектор размера N с текущей известной меткой времени, происходящего на каждом из процессов:

1. Когда хочу создать событие:
   - Добавлю в событие метку, соответствующую моим текущим часам: $L := C[i]$
   - Атомарно инкрементирую свои часы на единицу: $C[i] := C[i] + 1$
2. Когда получаю чужое событие:
   - Обновляю свои часы: $\forall j : C[j] := \max(C[j], C_m[j])$
   - Атомарно инкрементирую свои часы на единицу: $C[i] := C[i] + 1$

Иллюстрация векторных часов



Минимальность

Векторные часы — минимальное точное представление частичного порядка в общем случае (Charron-Bost, "Concerning the size of clocks", 1990)

- Causal consistency
- CRDT
- Отслеживание конфликтов
- Distributed debugging

Итоги

- Время – то, что позволяет нам рассуждать о событиях внутри распределенной системы
- В зависимости от порядка событий мы можем говорить о разных состояниях системы
- Физическое время – привычно нам, но оно не обладает монотонностью
- Логическое время – относительная конструкция, но с ней можно рассуждать о причинности

Блогопосты, чтобы разобраться лучше

1. Kevin Sookocheff. Lamport Clocks
2. Martin Fowler. Lamport Clocks
3. GeeksforGeeks. Lamport's Logical Clock
4. Michael Whittaker. Lamport's Logical Clocks
5. Kevin Sookocheff. Vector Clocks
6. Balraja Subbiah. Lamport Clocks and Vector Clocks